

Manual de Git y GitHub

Para instructores y alumnos del Diplomado ML en Seguros

Diplomado	Machine Learning en Seguros
Institucion	Facultad de Ciencias, UNAM
Modulo	1 — Introduccion a Python
Sesion	2 — GitHub y Control de Versiones
Instructores	Eric · Eduardo · Karla
Version	1.0

Este manual tiene dos partes: (1) como instalar Git en cualquier sistema operativo, y (2) como esta organizado el repositorio del diplomado — el flujo de trabajo para el instructor y para cada alumno.

Contenido

1.	Instalar Git — Windows, macOS y Linux	2
2.	Configurar Git por primera vez	3
3.	Arquitectura del curso: repositorios	4
4.	Para el instructor: repositorio maestro (oculto)	5
5.	Para los alumnos: su propio repositorio	6
6.	Flujo de trabajo semanal (sesion a sesion)	8
7.	El instructor revisa el trabajo del alumno (oculto)	9
8.	Comandos de referencia rapida	10

1. Instalar Git

Git es el software de control de versiones que corre en tu computadora. Debe instalarse ANTES de usar cualquier comando git o GitHub. Es independiente de Python y de Anaconda.

1.1 Windows

La forma mas sencilla es descargar el instalador oficial de git-scm.com.

Ir al sitio oficial

- 1 Abre tu navegador y ve a: git-scm.com/download/win — la descarga inicia automaticamente para tu arquitectura (64-bit en la mayoría de los casos).

Ejecutar el instalador

- 2 Doble clic en el archivo .exe descargado. Acepta todas las opciones por defecto — son adecuadas para la mayoría de los usuarios. No necesitas cambiar nada.

Opcion importante: editor por defecto

- 3 Cuando pregunte "Choosing the default editor": selecciona Visual Studio Code si lo tienes instalado. Si no, deja Vim (puedes cambiarlo despues).

Opcion PATH — muy importante

- 4 "Adjusting your PATH environment": selecciona "Git from the command line and also from 3rd-party software". Esto es fundamental para que git funcione en PowerShell y en Anaconda Prompt.

Finalizar

- 5 Clic en "Install" y espera 1-2 minutos. Al terminar, clic en "Finish".

✓ **Verificacion:** Abre Anaconda Prompt (o PowerShell) y escribe: `git --version` | Resultado esperado: `git version 2.x.x`

⚠ **Importante:** Si `git --version` no funciona despues de instalar: cierra y vuelve a abrir la terminal. El PATH nuevo solo aplica a terminales abiertas despues de la instalacion.

1.2 macOS

macOS viene con una version antigua de Git preinstalada. Lo mas sencillo es instalar la version actualizada con Homebrew o con las herramientas de desarrollo de Apple.

```
# Opcion A: Xcode Command Line Tools (mas sencillo, sin instalar nada extra)
xcode-select --install
# Aparece una ventana emergente - clic en 'Install'
# Tarda 5-10 minutos

# Verificar:
git --version    # git version 2.x.x (Apple Git)

# Opcion B: Homebrew (recomendado si ya tienen Homebrew instalado)
brew install git

# Verificar:
git --version    # git version 2.x.x
```

 **Tip:** Si no saben si tienen Homebrew: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`

1.3 Linux (Ubuntu / Debian)

```
# Una sola linea:
sudo apt update && sudo apt install git -y

# Verificar:
git --version
```

1.4 Verificacion final (todos los sistemas)

```
# Este comando debe funcionar en cualquier terminal despues de instalar
git --version
# Resultado esperado: git version 2.43.x (o mayor)

# Si no funciona en Windows, prueba desde:
# Inicio → buscar 'Git Bash' → abrir Git Bash → git --version
```

2. Configurar Git por Primera Vez

Esta configuracion solo se hace una vez en cada computadora. Le dice a Git quien eres — tu nombre y correo aparecern en cada commit que hagas.

```
# Configurar nombre (usa tu nombre real)
git config --global user.name "Maria Garcia Lopez"

# Configurar correo (usa el mismo correo con el que te registraste en GitHub)
git config --global user.email "maria.garcia@correo.com"

# Configurar la rama principal como 'main' (estandar actual)
git config --global init.defaultBranch main

# Configurar el editor de mensajes de commit (opcional)
# Si tienen VS Code:
git config --global core.editor "code --wait"

# Verificar que quedo guardado:
git config --global --list
# Deberia mostrar:
# user.name=Maria Garcia Lopez
# user.email=maria.garcia@correo.com
# init.defaultBranch=main
```

Nota: Esta configuracion es global: aplica a todos los repositorios de tu computadora. No necesitas repetirla por cada proyecto.

Crear cuenta en GitHub (si aun no la tienen)

1**Ir a github.com**

Clic en 'Sign up' (esquina superior derecha).

2**Completar el registro**

Correo → contraseña → nombre de usuario. Elige un usuario profesional, aparecera en tu CV: nombre-apellido o iniciales.

3**Verificar el correo**

GitHub envia un codigo de 6 digitos. Revisalo en tu bandeja de entrada (y en spam).

4**Plan gratuito**

Selecciona el plan 'Free'. Incluye repositorios privados ilimitados.

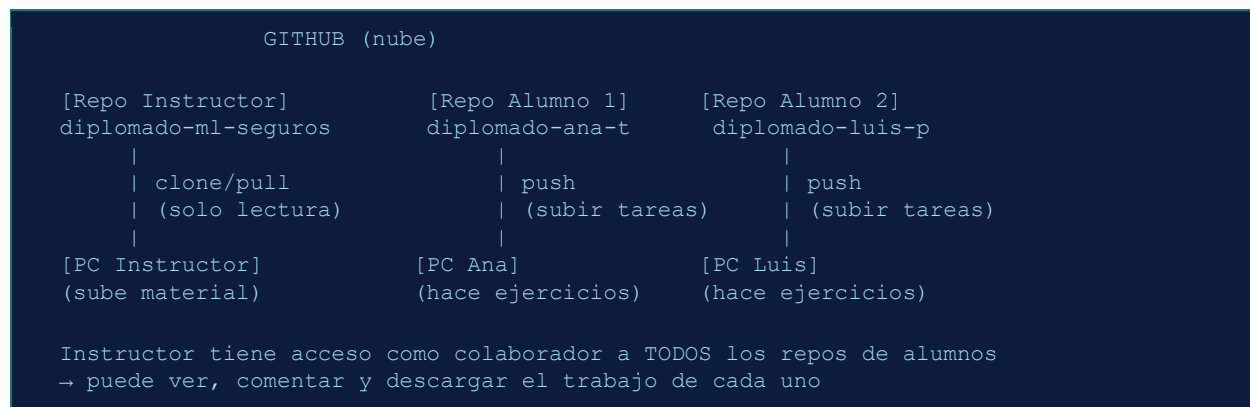
3. Arquitectura del Curso: Repositorios

El diplomado usa DOS tipos de repositorio con roles distintos. Es importante entender esta separacion antes de ejecutar cualquier comando.

Repositorio del INSTRUCTOR	Repositorio del ALUMNO	
diplomado-ml-seguros	diplomado-nombre-alumno	
Propietario: Eric (instructor)	Propietario: el alumno	
Contiene notebooks de clase, slides, datasets, requirements.txt	Contiene los ejercicios y tareas del alumno	
Los alumnos solo leen (clone/pull) — NO pueden subir cambios	El alumno sube su trabajo (push). El instructor puede revisar.	
Se actualiza cada sesion con material nuevo	Crece semana a semana con los ejercicios del alumno	
Publico o privado con alumnos como colaboradores de solo lectura	Privado, con el instructor como colaborador para revision	

Tip: Analogia: el repositorio del instructor es como el portal del curso (Moodle/Canvas) — todos lo consultan pero solo el profesor sube cosas. El repositorio del alumno es como su carpeta personal de tareas — solo el alumno y el profesor tienen acceso.

Diagrama del flujo



5. Para los Alumnos: Su Propio Repositorio

Cada alumno crea su propio repositorio privado para subir sus ejercicios y tareas. El instructor tiene acceso como colaborador para poder revisar el trabajo.

5.1 Crear el repositorio personal

Crear repositorio en GitHub

- 1 En github.com → boton '+' → 'New repository'. Nombre sugerido: diplomado-ml-TuNombre (ej: diplomado-ml-ana-torres). Visibilidad: Private. Marca 'Add a README file'. Clic en 'Create repository'.

Agregar al instructor como colaborador

- 2 En tu repositorio → Settings → Collaborators and teams → Add people → buscar el usuario de GitHub del instructor → rol: Write (para que pueda dejar comentarios y corregir).

Clonar a tu computadora

- 3 Copia la URL HTTPS de tu repositorio (boton verde 'Code'). En tu terminal: git clone `https://github.com/TuUsuario/diplomado-ml-TuNombre.git`

Crear la estructura de carpetas

- 4 Dentro de la carpeta del repositorio, crea subcarpetas por modulo y sesion. Esto facilita que el instructor encuentre tu trabajo.

Estructura recomendada del repositorio del alumno

```
diplomado-ml-ana-torres/  
├── README.md           # Tu nombre, grupo, correo  
├── modulo1/  
│   ├── sesion1_ejercicio_ana.ipynb  
│   ├── sesion2_ejercicio_ana.ipynb  
│   ├── sesion3_ejercicio_ana.ipynb  
│   └── ...  
├── modulo2/  
│   └── ...  
└── tareas/  
    ├── tarea1_ana.ipynb  
    └── tarea2_ana.ipynb
```

5.2 Primeros comandos del alumno

```
# 1. Clonar el repositorio del instructor (para obtener el material de clase)  
git clone https://github.com/InstructorUsuario/diplomado-ml-seguros.git
```

```
# 2. Clonar tu propio repositorio (para subir tus ejercicios)
git clone https://github.com/TuUsuario/diplomado-ml-TuNombre.git

# Ahora tienes DOS carpetas en tu computadora:
# diplomado-ml-seguros/      ← material de clase (NO modificar)
# diplomado-ml-TuNombre/    ← tus ejercicios (aquí trabajas)

# Flujo típico después de una sesión:
# 1. Copia el ejercicio de la clase a TU carpeta
# 2. Resuelve el ejercicio
# 3. Sube tu trabajo:
cd diplomado-ml-TuNombre
git add sesion2_ejercicio.ipynb
git commit -m "Sesión 2: ejercicio de tipos de datos completado"
git push
```

⚠ Importante: No subas archivos al repositorio del instructor. Solo tienes permiso de lectura. Todos tus ejercicios van en TU propio repositorio.

6. Flujo de Trabajo Semanal (Sesion a Sesion)

Este es el ciclo que se repite cada sesion del diplomado. Tanto el instructor como los alumnos tienen acciones especificas antes, durante y despues de cada clase.

Antes de cada sesion — Instructor

```
# Preparar y subir el material de la sesion
cd diplomado-ml-seguros

# Crear la carpeta de la sesion
mkdir -p modulo1/sesion4_funciones_modulos

# Copiar los archivos preparados (slides, notebook, ejercicio)
# ... copias los archivos a la carpeta ...

# Subir a GitHub
git add .
git commit -m "Sesion 4: agrega notebook de funciones y modulos"
git push

# Los alumnos hacen git pull para obtener el material
```

Antes de cada sesion — Alumno

```
# Descargar el material nuevo que subio el instructor
cd diplomado-ml-seguros
git pull
# Ahora tienes la carpeta sesion4 con el notebook del dia

# NUNCA hagas cambios en esta carpeta
# Si quieres practicar, copia el notebook a TU repositorio
```

Durante la sesion — Alumno

```
# Trabaja en TU repositorio, no en el del instructor
cd diplomado-ml-TuNombre

# Copia el ejercicio de la sesion a tu carpeta
# (hazlo manualmente en el explorador de archivos)

# Abre VS Code en tu carpeta
code .

# Resuelve el ejercicio en el notebook
```

Despues de cada sesion — Alumno

```
# Subir el ejercicio completado a tu repositorio
cd diplomado-ml-TuNombre
```



```
# Ver que archivos cambiaron
git status

# Agregar el ejercicio
git add modulol1/sesion2_ejercicio_ana.ipynb
# O agregar todo: git add .

# Commit con mensaje descriptivo
git commit -m "Sesion 2: ejercicio tipos de datos completado"

# Subir a GitHub
git push

# El instructor puede ver tu trabajo en github.com/TuUsuario/diplomado-ml-TuNombre
```

 **Tip:** Buenas practicas para mensajes de commit: usa verbos en presente y describe QUE hiciste. Ejemplos: 'Sesion 3: ejercicio de listas y diccionarios' · 'Tarea 1: analisis de cartera GMM' · 'Corrige calculo de prima mensual'

8. Comandos de Referencia Rapida

Guarda esta pagina — son los comandos que usaras el 90% del tiempo en el diplomado.

Comandos del instructor (subir material)

```
cd diplomado-ml-seguros      # entrar a la carpeta del curso
git status                   # ver que cambio
git add .                    # agregar todos los cambios
git commit -m "descripcion"  # guardar punto de control
git push                     # subir a GitHub
```

Comandos del alumno (descargar material y subir tareas)

```
# Obtener material nuevo del instructor:
cd diplomado-ml-seguros
git pull

# Subir tu ejercicio:
cd diplomado-ml-TuNombre
git add modulo1/sesion_N_ejercicio.ipynb
git commit -m "Sesion N: ejercicio completado"
git push
```

Tabla de comandos esenciales

Comando	Que hace
git --version	Verificar que Git esta instalado
git config --global user.name "..."	Configurar nombre (una sola vez)
git config --global user.email "..."	Configurar correo (una sola vez)
git clone <url>	Descargar un repositorio de GitHub a tu PC
git status	Ver que archivos cambiaron
git add .	Preparar todos los cambios para el commit
git add archivo.ipynb	Preparar un archivo especifico
git commit -m "msg"	Guardar un punto de control con descripcion
git push	Subir commits locales a GitHub
git pull	Descargar cambios recientes de GitHub
git log --oneline	Ver historial de commits resumido
git diff	Ver exactamente que lineas cambiaron

Errores comunes y soluciones

Error	Solucion
<code>'git' no se reconoce como comando</code>	Cierra y vuelve a abrir la terminal despues de instalar Git. En Windows usa Git Bash o Anaconda Prompt.
<code>Permission denied (publickey)</code>	Usa HTTPS en lugar de SSH para la URL del repositorio. Formato: <code>https://github.com/usuario/repo.git</code>
<code>Authentication failed</code>	GitHub ya no acepta contraseñas. Genera un Personal Access Token (PAT) en Settings → Developer settings → Tokens.
<code>fatal: not a git repository</code>	No estas dentro de una carpeta con repositorio Git. Navega con <code>cd</code> a la carpeta correcta.
<code>error: failed to push some refs</code>	Hay cambios en GitHub que no tienes localmente. Haz primero: <code>git pull</code> y luego intenta <code>git push</code> de nuevo.